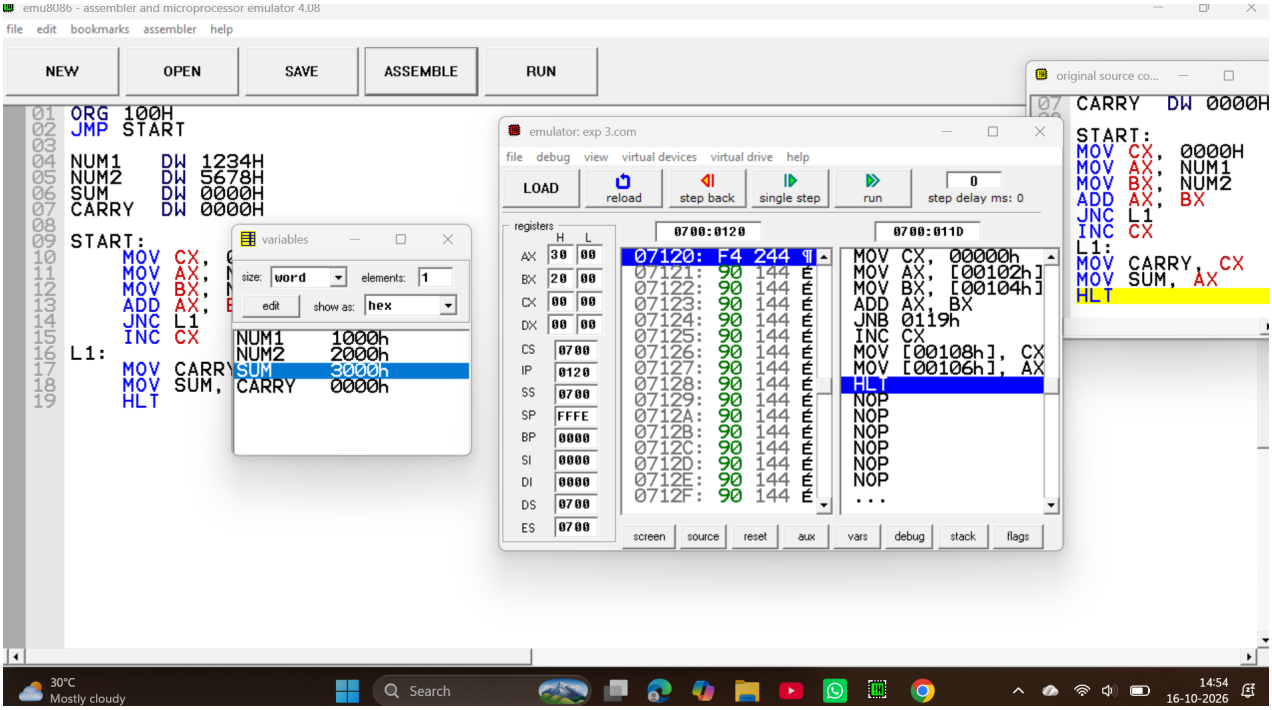
Multi-Byte Data and Array Operations Using 8086 Indirect Addressing

Major Topics / Minor Topics Included

- Double-precision / 32-bit addition
- Multi-byte data processing
- Sum of N numbers
- Array traversal
- Indirect addressing
- SI and DI registers
- Memory data manipulation
- Loop-based processing

Reference Screenshot



8086 Assembly Program – 16-bit Addition with Carry

The screenshot shows an 8086 emulator program that adds two 16-bit values and stores the result and carry.

```
ORG 100H
JMP START

NUM1  DW 1234H
NUM2  DW 5678H
SUM   DW 0000H
CARRY DW 0000H

START:
    MOV CX, 0000H
    MOV AX, NUM1
    MOV BX, NUM2
    ADD AX, BX
    JNC L1
    INC CX
L1:
    MOV CARRY, CX
    MOV SUM, AX
    HLT
```

Explanation

• **ORG 100H** – sets the program's starting offset for a .COM-style program.

• **JMP START** – jumps over the data declarations to the executable instructions.

• **NUM1 / NUM2** – two 16-bit input values stored as words.

• **MOV AX, NUM1** and **MOV BX, NUM2** – load the two values into registers.

• **ADD AX, BX** – adds the values; the result is in AX and the carry flag is updated.

• **JNC L1** – jumps if no carry occurred. If carry occurred, **INC CX** sets CX to 1.

• **MOV CARRY, CX** – stores the carry indicator (0 or 1).

• **MOV SUM, AX** – stores the low 16 bits of the sum.

• **HLT** – halts execution.

Result for the values shown

1234H + 5678H = 68ACH. Carry = 0, so SUM = 68ACH and CARRY = 0000H.

Note: The screenshot's variables window appears to show addresses/values in a different view than the source declarations. The expected arithmetic result above is based on the source code values.

Exam Notes: How This Relates to the Listed Topics

Double-precision / 32-bit addition: Add the lower 16-bit words first using ADD, then add the upper words with ADC so the carry from the lower addition is included.

Multi-byte data processing: Process data in 8-bit or 16-bit chunks. Carry/borrow flags help connect operations across chunks.

Sum of N numbers: Use a loop to read each element, add it to an accumulator, and advance the pointer to the next element.

Array traversal: SI is commonly used as a source index/pointer; increment it by 2 when traversing a word array.

Indirect addressing: For example, MOV AX, [SI] reads the word from the memory address stored in SI.

SI and DI registers: SI (Source Index) and DI (Destination Index) are index registers often used for arrays and memory copies.

Memory data manipulation: MOV reads/writes values; ADD/ADC perform arithmetic; INC updates counters or pointers.

Loop-based processing: Initialize CX with the number of elements, process one element per iteration, and use LOOP to repeat until CX reaches zero.

Example: Sum an array of 5 words using SI

```
ORG 100H
JMP START

ARR DW 0001H, 0002H, 0003H, 0004H, 0005H
SUM DW 0000H

START:
    MOV SI, OFFSET ARR
    MOV CX, 0005H
    XOR AX, AX

NEXT:
    ADD AX, [SI]
    ADD SI, 2
    LOOP NEXT

    MOV SUM, AX
    HLT
```

Expected result: 0001H + 0002H + 0003H + 0004H + 0005H = 000FH. AX and SUM contain 000FH after execution.